

Authentication Module

Talabaty Spring Boot Backend - Implementation and API Reference

Capability	Implemented behavior
Credentials	Email and BCrypt-verified password
Email verification	Login is rejected when emailVerified is false
Tokens	JWT access token only - no refresh tokens
JWT claims	userId, email, role
Rate limit	5 failed attempts per email-and-IP key within 15 minutes

Module scope

This module authenticates a user with email and password, verifies the stored BCrypt hash, requires a verified email address, and issues a signed access JWT. It uses stateless Spring Security.

Key security decisions

Only access tokens are issued. HTTP Basic authentication, form login, and server sessions are disabled. A valid Bearer token is required for all endpoints except authentication and Swagger routes.

Login API

Endpoint

POST /api/auth/login

Request body

```
{
  "email": "customer@example.com",
  "password": "TestPassword123!"
}
```

Validation

Field	Rules
email	Required and must use a valid email format
password	Required and cannot be blank

Successful response - 200 OK

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "userId": 1,
  "email": "customer@example.com",
  "role": "CUSTOMER"
}
```

The access token is valid for 900 seconds (15 minutes). The shared LoginResponse class also contains a message field, which is not used by the login flow.

Errors and Rate Limiting

Status	When it is returned	Response detail
400	Invalid JSON, missing fields, invalid email format	Request validation failed
401	Unknown email or incorrect password	Invalid email or password
403	Correct credentials but email is not verified	Email is not verified
429	Rate limit exceeded	Too many failed login attempts. Please try again later.
500	Unexpected authentication error	An unexpected authentication error occurred

Validation error example

```
{
  "status": 400,
  "detail": "Request validation failed",
  "errors": {
    "email": "Email must be valid",
    "password": "Password is required"
  }
}
```

Rate-limit behavior

The first five failed password attempts return 401. The next attempt for the same normalized email and direct client IP returns 429. A successful verified login clears the stored failed-attempt count.

In-memory limitation

Counters reset when the backend restarts and are not shared between backend instances. For a multi-instance production deployment, replace this with Redis or gateway-backed rate limiting. If the application is deployed behind a reverse proxy, configure trusted forwarded-IP handling.

Security Architecture

Class	Responsibility
UserRepository	Retrieves a user with findByEmail
JwtProperties	Reads the Base64 JWT secret and token lifetime
JwtService	Creates and validates HS256 access JWTs
JwtAuthenticationFilter	Reads Authorization: Bearer tokens on protected requests
SecurityConfig	Registers BCrypt, stateless security, public paths, and the JWT filter
LoginRateLimitProperties	Reads failed-attempt count and time-window configuration
LoginRateLimitService	Tracks failures in memory by email and client IP
AuthServiceImpl	Coordinates validation, credentials, verification, rate limiting, and JWT creation
AuthExceptionHandler	Returns consistent ProblemDetail responses for auth failures

JWT-protected requests

Authorization: Bearer

Public routes: /api/auth/**, /swagger-ui/**, and /v3/api-docs/**. All other routes require a valid access JWT.

Configuration

```
security.jwt.secret=  
security.jwt.access-token-expiration-seconds=900  
  
security.login-rate-limit.max-failed-attempts=5  
security.login-rate-limit.window-seconds=900
```

Testing and Swagger

Swagger UI

```
http://localhost:8080/swagger-ui/index.html
```

Open Authentication, select POST /api/auth/login, choose Try it out, and submit a request. localhost can be opened only on the same laptop that is running the Spring Boot application.

Development test accounts

Purpose	Email	Password	Expected result
Verified login	test.customer@talabaty.local	TestPassword123!	200 with access token
Unverified login	unverified.customer@talabaty.local	TestPassword123!	403
Wrong password	test.customer@talabaty.local	WrongPassword456!	401

Rate-limit test

Send the wrong-password request five times. Send it once more from the same laptop: the API should return 429. Restart the application to reset its in-memory counters during development.

Production checklist

- Move the JWT secret and database password from application.properties into environment variables or a secrets manager.
- Replace the development JWT secret before deployment because it was exposed during development setup.
- Restrict CrossOrigin origins to the real frontend domain.
- Add automated tests for validation, credentials, unverified email, rate limiting, and protected routes.
- Use Redis or an API gateway for rate limiting when multiple backend instances are deployed.